



Monad

Findings & Analysis Report

2026-02-26

Table of contents

- [Overview](#)
- [About C4](#)
- [Summary](#)
- [Scope](#)
- [Severity Criteria](#)
- [High Risk Findings \(4\)](#)
 - [\[H-01\] Block policy discounts gas price by incorrectly applying EIP-1559 to legacy transactions](#)
 - [\[H-02\] Attacker can send malicious EIP-7702 transactions that cause rounds to time out and halt the chain](#)
 - [\[H-03\] Incorrect affordability checks admits invalid transactions allowing txpool and block building DoS](#)
 - [\[H-04\] EIP-7702 order dependent delegated-status mismatch enables persistent free chain DoS](#)
- [Medium Risk Findings \(7\)](#)
 - [\[M-01\] Lack of authorization in RaptorCast Secondary protocol group messages allows arbitrary node impersonation leading to DoS](#)
 - [\[M-02\] Bounded channel panic in TokioTaskUpdater causes node crash leading to realistic chain halt](#)
 - [\[M-03\] Remote process crash \(OOM\) via post-serialization size check and large batch aggregation in JSON-RPC](#)
 - [\[M-04\] Incorrect Write Position in Block Device Trim Operation](#)
 - [\[M-05\] Typed receipt encoding does not conform to standard Ethereum RPC format](#)
 - [\[M-06\] Vote timer callback uses potentially stale timer round instead of actual vote round, causing vote misrouting](#)
 - [\[M-07\] Consensus validator accepts blocks with mismatched basefee and execution ``basefeepergas``](#)
- [Low Risk and Informational Issues](#)
 - [L-01 HTTP Host label unsanitized → unbounded metrics cardinality \(Invariant: OpSec\)](#)
 - [L-02 JSON-RPC returns internal error details → information disclosure \(Invariant: OpSec\)](#)
- [Disclosures](#)

Overview

About C4

Code4rena (C4) is a competitive audit platform where security researchers, referred to as Wardens, review, audit, and analyze codebases for security vulnerabilities in exchange for bounties provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the Monad smart contract system. The audit took place from September 15 to October 12, 2025.

Final report assembled by Code4rena.

Summary

The C4 analysis yielded an aggregated total of 11 unique vulnerabilities. Of these vulnerabilities, 4 received a risk rating in the category of HIGH severity and 7 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 21 QA reports compiling issues with a risk rating of LOW severity or informational.

All of the issues presented here are linked back to their original finding, which may include relevant context from the judge and Monad team.

Scope

The code under review can be found within the [C4 Monad repository](#), and is composed of 718 files written in Rust and C++ programming languages and includes 164,246 lines of Rust and C++ code.

The code in C4's Monad repository was pulled from:

- Repository: <https://github.com/category-labs/monad>
- Commit hash: `020cb9ed751480cf79ad334ab36a6ecd606235e7`
- Repository: <https://github.com/category-labs/monad-bft>
- Commit hash: `6aacc2ec51eac66a2cf18ec64ab0df2c1448990`

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/informational.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges
- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on [the C4 website](#), specifically our section on [Severity Categorization](#).

High Risk Findings (4)

[H-01] Block policy discounts gas price by incorrectly applying EIP-1559 to legacy transactions

Submitted by [RadiantLabs](#), also found by [Ox15](#) and [OxAsen](#)

`bft/monad-eth-block-policy/src/lib.rs` [#L77](#)

Block policy implements static tracking of account balances and nonce updates to allow consensus to lead execution by `k` blocks, by allowing *only valid* transactions to be included in a block.

How it does it is by tracking the “worst case balance” and “nonce” of each account sending transactions and signing EIP-7702 authorizations.

In the case of the “worst case balance” calculation, there is a miscalculation in the `compute_txn_max_gas_cost`

function, where the `base_fee` ceiling is applied to transactions that are not EIP-1559 (aka legacy transactions):

```
File: bft/monad-eth-block-policy/src/lib.rs
77: pub fn compute_txn_max_gas_cost(txn: &TxEnvelope, base_fee: u64) -> U256 {
78:     let gas_limit = U256::from(txn.gas_limit());
79:     let max_fee = U256::from(txn.max_fee_per_gas());
80:     let priority_fee = U256::from(txn.max_priority_fee_per_gas().unwrap_or(0));
81:     let base_fee = U256::from(base_fee);
82:     let gas_bid = max_fee.min(base_fee.saturating_add(priority_fee));
83:     gas_limit.checked_mul(gas_bid).expect("no overflow")
84: }
```

This contrasts with the execution layer which instead excludes legacy transactions from the EIP-1559 gas ceiling:

```
File: bft/monad-cxx/monad-execution/category/execution/ethereum/transaction_gas.cpp
144: inline constexpr uint256_t priority_fee_per_gas(
145:     Transaction const &tx, uint256_t const &base_fee_per_gas) noexcept
146: {
147:     MONAD_ASSERT(tx.max_fee_per_gas >= base_fee_per_gas);
148:     auto const max_priority_fee_per_gas = tx.max_fee_per_gas - base_fee_per_gas;
149:
150:     if (tx.type == TransactionType::eip1559 ||
151:         tx.type == TransactionType::eip4844 ||
152:         tx.type == TransactionType::eip7702) {
153:         return std::min(tx.max_priority_fee_per_gas, max_priority_fee_per_gas);
154:     }
155:     // EIP-1559: "Legacy Ethereum transactions will still work and
156:     // be included in blocks, but they will not benefit directly from
157:     // the new pricing system. This is due to the fact that upgrading
158:     // from legacy transactions to new transactions results in the
159:     // legacy transaction's gas_price entirely being consumed either
160:     // by the base_fee_per_gas and the priority_fee_per_gas."
161:     return max_priority_fee_per_gas;
162: }
```

As a result of the bug in the consensus logic, the gas cost of a transaction can be underestimated, allowing transactions to be included in a block when the sender's balance is not sufficient to guarantee cover for the gas fees.

When these transactions reach execution, the C++ logic will mark them invalid, eventually invalidating the whole block (because transactions that revert are acceptable in a valid block but invalid transactions aren't).

Apart from the DoS scenario of invalid transactions poisoning blocks, this vulnerability also opens up a scenario where malicious actors can have consensus and include an arbitrarily high number of transactions without paying any fees, thus circumventing the economic barrier of gas that protects the Monad nodes' infrastructure from abuse.

🔗 Recommended mitigation steps

Fix block policy to calculate legacy transaction gas cost as `gas_limit * gas_price` like done in execution.

[View detailed Proof of Concept](#)

🔗 [H-02] Attacker can send malicious EIP-7702 transactions that cause rounds to time out and halt the chain

Submitted by [TheSchnilch](#), also found by [OxAsen](#) and [octOpwn](#)

- `bft/monad-eth-tpool/src/pool/transaction.rs` [#L112-L125](#)
- `bft/monad-eth-block-validator/src/lib.rs` [#L467-L475](#)

The root cause is that during transaction validation in the mempool, the `chain_id` of an authorization is checked before verifying whether the authority is the `SYSTEM_SENDER_ETH_ADDRESS` (see first GitHub link). If the `chain_id` is incorrect, the transaction itself is still considered valid, but the authorization is not.

During block validation, however, the checks are performed in the opposite order: the `SYSTEM_SENDER_ETH_ADDRESS` check is done first, followed by the `chain_id` check (see second GitHub link). If the `SYSTEM_SENDER_ETH_ADDRESS` check fails, the whole transaction and therefore the block is invalid.

The problem arises when someone sends a transaction with the authority set to `SYSTEM_SENDER_ETH_ADDRESS` but with an invalid `chain_id`. In this case:

- Mempool validation: The transaction is accepted, because the `chain_id` check fails first, so the `SYSTEM_SENDER_ETH_ADDRESS` check is never performed.
- Block validation: The transaction causes an error, because the `SYSTEM_SENDER_ETH_ADDRESS` check runs first, and the block is therefore deemed invalid and not voted on: [see here](#). This means the block is never added to the block tree and therefore [cannot be coherent](#). Because `try_vote` would only be called in `try_add_and_commit_blocktree`, it will never vote on the block, and a [timeout will occur](#).

If an attacker sends such a transaction to many nodes, each node may include it in a block, causing repeated timeouts. Because the leader schedule is known, the attacker could also always send their transactions to the next leaders. This means he does not have to send the transaction to all nodes. This process can be repeated indefinitely, effectively halting the chain because no blocks are successfully executed. Since the transaction is never executed, the attacker can resubmit it endlessly at no cost.

🔗 Recommended mitigation steps

The `SYSTEM_SENDER_ETH_ADDRESS` check should also be performed first during transaction validation in the mempool. This way, invalid transactions would be rejected early and never included in a block, allowing the chain to continue operating normally.

[View detailed Proof of Concept](#)

🔗 [H-03] Incorrect affordability checks admits invalid transactions allowing txpool and block building DoS

Submitted by [octOpwn](#), also found by [OxAsen](#) and [lian886](#)

- `bft/monad-eth-tpool/src/pool/mod.rs` [#L184-L197](#)
- `bft/monad-eth-block-policy/src/lib.rs` [#L77-L84](#)
- `bft/monad-eth-block-policy/src/lib.rs` [#L541-L556](#)

Transaction's priority fee is ignored in txpool admission and only the base-fee is used to check affordability. However, proposal-time validation uses the full EIP-1559 gas bid. So an attacker can stuff the pool with transactions that pass the insert-time gate but are impossible to include.

During proposal, these fail `try_add_transaction(...)`; they remain in the txpool and will be selected again on subsequent proposals. The pool can be filled with similar high-tip transactions causing empty blocks due to no other transactions being admitted into it, causing transaction inclusion to stop completely once transactions are forwarded or submitted to all 200 leaders.

Important! This is a separate issue and root cause from my other txpool stuffing issue: this one is due to `gas_bid` being checked incorrectly, the other is replacement-checks + nonce-gaps + eviction/promotion rules.

🔗 Impact

When all 200 leaders are affected (either due to forwarding or direct submissions):

- Block building starvation: proposals contain no txs because tracked is stuffed by high-tip transactions from senders with insufficient balances.
- Txpool admission DoS: attacker transactions fill up both pools and can never be included (insufficient balance) which prevents admission of other transactions due to limited capacity.

Root Cause

- Insert-time affordability check only uses base fee for balance check: `balance ≥ base_fee × gas_limit`.
- Proposal-time validation uses full EIP-1559 bid including max priority fee.
- Invalid transactions are returned to the tracked pool instead of being evicted to pending.
- No affordability-based pruning post-insert; eviction is time-based only, and easily bumped by replacement.

Affected Code

- Insert-time affordability (base_fee-only gate):
 - Balance vs base fee × gas limit in `insert_txs(...)`:
 - <https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-eth-txpool/src/pool/mod.rs#L184-L197>
- Proposal-time affordability (full gas bid):
 - Gas bid computation `compute_txn_max_gas_cost(...)`:
 - <https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-eth-block-policy/src/lib.rs#L77-L84>
 - `EthBlockPolicyBlockValidator::try_add_transaction(...)` enforces affordability at bid and reserve:
 - TFM-disabled path ($\text{max_fee} \times \text{gaslimit}$): <https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-eth-block-policy/src/lib.rs#L541-L556>
 - TFM-enabled emptying path (checks bid, value, reserve): <https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-eth-block-policy/src/lib.rs#L574-L599>
- Sequencer sampling and skip-on-failure (no backfill):
 - Main proposal loop and selection:
 - loop and pop: <https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-eth-txpool/src/pool/tracked/sequencer.rs#L175-L185>
 - add-to-proposal gate; returns false on validator error: <https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-eth-txpool/src/pool/tracked/sequencer.rs#L287-L296>
- Forwarding gate ignores bid (base_fee-only):
 - `ValidEthTransaction::get_if_forwardable(...)` checks `max_fee ≥ base_fee`:
 - <https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-eth-txpool/src/pool/transaction.rs#L260-L268>

Preconditions

While no fees are paid (transactions aren't included), the attacker needs to fund addresses:

- They need one address per `MAX_ADDRESSES` for Tracking pool.
- So ~16k addresses with ~0.0021 MON each (21k min gas @ 100 gwei = 0.0021).
- Total~=34 MON. At 100B total supply and 20B valuation, this is roughly \$7 in pre-funding.

Attack path

1. Submit one high-tip EIP-1559 tx for each funded address, for each leader, with `gas_bid = base_fee + high-tip`. Transactions will never be included, this costs no money.

2. `insert_txs(...)` admits them (base-fee-only check).
3. Sequencer samples up to `tx_limit` addresses.
4. `try_add_transaction(...)` rejects many on gas-bid affordability; those sampled addresses are skipped for this proposal, remain in the pool, and will be selected again.
5. Blocks are constructed empty, validators lose fees.

🔗 Recommended mitigation steps

- Tighten insert-time gate: use full gas-bid affordability (`compute_txn_max_gas_cost`) for admission.
- Evict invalid transactions to pending when they fail static checks.
- Gate forwarding on affordability at current gas bid.

[View detailed Proof of Concept](#)

🔗 [H-04] EIP-7702 order dependent delegated-status mismatch enables persistent free chain DoS

Submitted by [octOpwn](#), also found by [OxPhantom](#), [DieEmpty](#), and [RadiantLabs](#)

- `bft/monad-eth-tpool/src/pool/tracked/sequencer.rs` [#L248-L253](#)
- `bft/monad-eth-block-policy/src/lib.rs` [#L1212-L1219](#)
- `bft/monad-eth-block-policy/src/lib.rs` [#L1347-L1354](#)

Txpool marks an authority as delegated in order dependent way - once it includes the EIP-7702-carrying transaction in the proposal (ordering-sensitive). However, block policy globally sets all recovered EIP-7702 authorities as delegated for the entire block before any per-transaction reserve checks (NOT-ordering-sensitive).

A user can exploit this mismatch by submitting:

- X: an EIP-1559 ($\leq 30M$ gas) transaction tuned to pass the emptying path ($\text{balance} \geq \text{txn_max_gas}$) but fail the non-emptying reserve-only path ($\text{txn_max_gas} > \text{max_reserve_balance}$), with a very high tip.
- Y: a valid-chain EIP-7702 transaction carrying an authorization for the same authority as X, with a slightly lower tip so it is included after X in the txpool's ordering.

Txpool admits X first under emptying rules, then adds Y and flips delegation. When proposed, consensus peers treat the authority as delegated globally for the whole block, reclassify X as non-emptying, and reject the block for `InsufficientReserveBalance`. Because the block fails pre-execution, the attacker pays no gas; the transactions remain in the pool and are re-proposed, enabling repeated DoS.

🔗 Impact

- Repeated, full, user-triggered DoS across leader slots; blocks built atop the invalid block are also rejected.
- Chain can stall nearly completely due to each proposer continually trying to build invalid blocks.
- Honest proposers/validators lose rewards/fees; attacker never pays gas (pre-execution rejection).
- Broken invariants:
 - “Chain finality: Speculative execution results of a previous block will be finalized within the designated system parameters if a supermajority of nodes are participating in consensus”. Finality is repeatedly prevented.
 - “Transactions: The total token deduction from a sender's balance must be equal to the sum of

value and the product of `gas_price` and `gas_limit`. A transaction's `max_expenditure` shall not exceed the sender's available balance at the time of consensus."

🔗 Affected Code

- Txpool proposal sequencing (delegation flips only after a 7702 tx is included):
 - Marks `is_delegated = true` only after including a 7702-carrying tx (ordering-dependent):
 - <https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-eth-txpool/src/pool/tracked/sequencer.rs#L248-L253>
- Block policy global pre-marking (ordering-insensitive):
 - Collects 7702 authorities up-front from all txs in the block (no ordering dependence here):
 - <https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-eth-block-policy/src/lib.rs#L1212-L1219>
 - Pre-marks all collected authorities as delegated before per-tx reserve checks:
 - <https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-eth-block-policy/src/lib.rs#L1347-L1354>

🔗 Root cause

- Semantic and timing mismatch for when `is_delegated` is derived:
 - Txpool: delegated status is applied lazily during proposal assembly as Y is included.
 - Block policy: delegated status is applied at once to all authorities present anywhere in the block before checking any tx.
- This reclassifies X between components: txpool admits X as emptying; policy re-evaluates X as non-emptying and rejects it under reserve rules.

🔗 Attack path

1. User submits two transactions targeting the same authority A:
 - X (EIP-1559): $\leq 30M$ gas, very high tip so `gas_at_bid * gas_limit > max_reserve_balance`; `value = 0`.
 - Y (EIP-7702): valid `chain_id`, contains an authorization signed by A, with a slightly lower tip so it is included after X in the txpool ordering.
2. Txpool proposal assembly selects X first (higher tip) while A is not yet marked delegated, admitting X under emptying rules (gas-only check).
3. Txpool then includes Y and marks A as delegated (ordering-dependent state flip) after Y's inclusion.
4. Block policy coherency validation collects 7702 authorities from the whole block up-front and pre-marks A as delegated for the entire block.
5. The same X is re-evaluated under non-emptying rules and fails the reserve-only check with `InsufficientReserveBalance`.
6. The block is rejected pre-execution; proposer loses the slot. Transactions stay in the pool and will be re-proposed. By repeating across leaders (optionally rotating senders), the attacker achieves persistent DoS and liveness degradation.

🔗 Recommended mitigation steps

Unify delegation semantics across components, apply the order-sensitive semantics at the consensus level.

[View detailed Proof of Concept](#)

🔗 Medium Risk Findings (7)

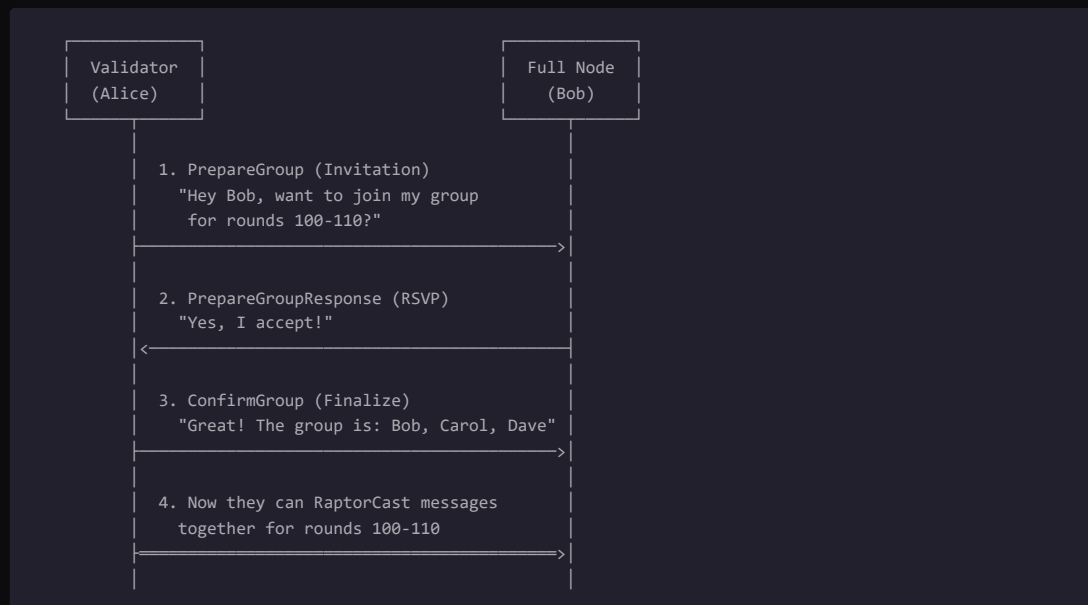
🔗 [M-01] Lack of authorization in RaptorCast Secondary protocol group messages allows arbitrary node impersonation leading to DoS

Submitted by [dontonka](#), also found by [OxAsen](#), [Oxvd](#), [KeccakCrew](#), [luncy](#), and [zca](#)

- `bft/monad-raptorcast/src/raptorcast_secondary/client.rs` [#L256-L283](#)
- `bft/monad-raptorcast/src/raptorcast_secondary/client.rs` [#L285-L393](#)
- `bft/monad-raptorcast/src/raptorcast_secondary/publisher.rs` [#L305-L330](#)

A vulnerability exists in the RaptorCast Secondary protocol's group management system where identity fields in messages are never validated against the cryptographic signature. While messages are cryptographically signed and the signature is verified in RaptorCast, the protocol fails to check that the recovered public key matches identity claims within the message payload (such as `validator_id`, `node_id`, etc.) at the application level which opens the door for multiple DoS attacks. This can essentially DoS the entire RaptorCast Secondary protocol, meaning fullnode would most likely need to fallback on using BlockSync in order to get blocks information.

The Protocol Flow



🔗 Attack Scenarios

This allows an attacker to:

- **PrepareGroup Impersonation:** impersonate any validator by sending a `PrepareGroup` message to fullnode with a victim's `validator_id`. This can be used to block legitimate validators from grouping with fullnode (group slot exhaustion DoS - `max_num_group` is only 3). Another angle would be to fill `pending_confirms` with fake information which would waste resources (CPU, Memory). Reputation attack is also a concern.

```
### Attack flow
1. Attacker generates own keypair (attacker_key)
2. Creates PrepareGroup message:
{
  validator_id: victim_validator_pubkey, # Impersonation!
  max_group_size: 10,
  start_round: 1000,
```

```
end_round: 1240
}
3. Signs message with attacker_key (NOT victim's key)
4. Sends to full nodes
5. Full nodes accept (no validation) and respond to ATTACKER
```

- **PrepareGroupResponse Injection:** impersonate any fullnode by sending `PrepareGroupResponse` messages to validator with a victim's `node_id`.

```
### Attack flow
```

- Attacker observes validator sending PrepareGroup (round 1000)
- Attacker generates 10 fake full node identities
- For each fake identity: Creates PrepareGroupResponse { req: PrepareGroup(validatorid=observed, startround=1000, ...), nodeid: fakefullnodepubkey, # Impersonation! accept: true } Signs with attackerkey (NOT fake fullnode's key) Sends to validator
- Validator processes all 10 responses (no validation)
- Validator attempts to create group with 10 members

- **ConfirmGroup Poisoning:** impersonate any validator by sending `ConfirmGroup` messages to fullnode with a victim's `validator_id`.

```
### Attack flow
```

- Attacker sends PrepareGroup as fake validator (rounds 1000-1240)
- Full node accepts and responds
- Attacker sends ConfirmGroup { prepare: PrepareGroup(validatorid=fakevalidator, ...), peers: [attackernode1, attackernode2, ...], # Malicious peers! namerecords: [fakerecordswithattacker_IPs] }
- Full node accepts ConfirmGroup (no validation)
- Full node updates peer discovery with attacker's IPs
- Full node connects exclusively to attacker infrastructure

🔗 Root Cause

The RaptorCast message handling performs two-stage validation:

```
Stage 1: Cryptographic Signature Validation ✅ IMPLEMENTED
└ Recovers public key from signature
└ Verifies signature is cryptographically valid
└ Result: "from" = recovered_pubkey

Stage 2: Identity Field Validation ❌ MISSING
└ Compare message.validator_id to "from"
└ Compare message.node_id to "from"
└ Result: NEVER PERFORMED
```

The vulnerability exists because Stage 2 is completely missing.

🔗 Impact

- **DoS legitimate traffic:** while there is resource waste involved in those DoS attacks, the main impact is they allow the disabling of the `RaptorCast Secondary protocol` functionality, creating noise, degradation and confusion in the chain in terms of block propagation.

🔗 Likelihood

HIGH

- Anyone can trigger this attack (see PoC), no need to be an official node in the chain (validator, full node, etc.), it simply requires connection to the node port, which is required to be open as this is how peers communicate together in Monad.
- There is no cost for this attack in terms of money, only very little resources are needed and very little effort to sustain it.

🔗 Recommended mitigation steps

All three vulnerabilities stem from missing identity validation. A single comprehensive fix addresses all impacts:

This is not a working version of the mitigation (will not compile), but gives a good idea of what I have in mind.

```
// Unified fix for all three message types
fn validate_group_message_identity(
    msg: &FullNodesGroupMessage,
    recovered_pubkey: &PublicKey,
    validator_set: &ValidatorSet,
) -> Result<(), ValidationError> {
    match msg {
        FullNodesGroupMessage::PrepareGroup(prepare) => {
            // Fix for Attack Vector 1
            if prepare.validator_id.pubkey() != recovered_pubkey {
                return Err(ValidationError::ValidatorIdMismatch);
            }
            if !validator_set.contains(&prepare.validator_id) {
                return Err(ValidationError::NotInValidatorSet);
            }
        }

        FullNodesGroupMessage::PrepareGroupResponse(resp) => {
            // Fix for Attack Vector 2
            if resp.node_id.pubkey() != recovered_pubkey {
                return Err(ValidationError::NodeIdMismatch);
            }
        }

        FullNodesGroupMessage::ConfirmGroup(conf) => {
            // Fix for Attack Vector 3
            if conf.prepare.validator_id.pubkey() != recovered_pubkey {
                return Err(ValidationError::ValidatorIdMismatch);
            }
            if !validator_set.contains(&conf.prepare.validator_id) {
                return Err(ValidationError::NotInValidatorSet);
            }

            // Validate peer list integrity
            if conf.peers.len() != conf.name_records.len() {
                return Err(ValidationError::PeerRecordMismatch);
            }
            for (peer, record) in conf.peers.iter().zip(&conf.name_records) {
                if record.recover_pubkey()? != *peer {
                    return Err(ValidationError::InvalidPeerSignature);
                }
            }
        }
    }
}

Ok(())
}
```

[View detailed Proof of Concept](#)

[\[M-02\] Bounded channel panic in TokioTaskUpdater causes node crash leading to realistic chain halt](#)

Submitted by [dontonka](#)

bft/monad-updaters/src/lib.rs [#L141-L161](#)

The [TokioTaskUpdater executor](#) uses a bounded channel with only [1024 slots](#) for command batches. When an attacker sends `ForwardedTx` messages that exceed this capacity, the executor panics with “executor is lagging” instead of gracefully handling the overflow. An unauthenticated attacker can crash any validator by sending `ForwardedTx` messages with `batches_per_conn × num_connections > 1024` (e.g., 500 batches × 5 connections = 2,500 batches). The vulnerability stems from using `.expect()` on `try_send()` rather than implementing proper backpressure or error handling.

```
#[cfg(feature = "tokio")]
impl<U, E> Executor for TokioTaskUpdater<U, E>
where
    U: Updater<E>,
    U::Command: Send + 'static,
    E: Send + 'static,
{
    type Command = U::Command;

    fn exec(&mut self, commands: Vec<Self::Command>) {
        self.verify_handle_liveness();

        self.command_tx
            .try_send(commands)
            .expect("executor is lagging")
    }

    fn metrics(&self) -> ExecutorMetricsChain {
        ExecutorMetricsChain::from(&self.metrics)
    }
}
```

[Impact](#)

- **Crash the node:** crash any discoverable node in seconds.
- **Chain halt:** very realistic scenario by simply killing all the validator sets in the current epoch.

[Likelihood](#)

- Anyone can trigger this attack (see PoC), no need to be an official node in the chain (validator, full node, etc.), it simply requires connection to the node port, which is required to be open as this is how peers communicate in Monad.
- There is no cost for this attack in terms of money, only very little resources are needed.

[Recommended mitigation steps](#)

Implement proper backpressure or error handling such that it doesn't crash.

[View detailed Proof of Concept](#)

[\[M-03\] Remote process crash \(OOM\) via post-serialization size check and large batch aggregation in JSON-RPC](#)

Submitted by [RadiantLabs](#), also found by [Almanax](#) and [johnyfwesh](#)

- `bft/monad-rpc/src/handlers/mod.rs` [#L129-L151](#)
- `bft/monad-rpc/src/handlers/mod.rs` [#L154-L169](#)
- `bft/monad-rpc/src/handlers/eth/txn.rs` [#L154-L181](#)
- `bft/monad-rpc/src/handlers/eth/gas.rs` [#L402-L466](#)
- `bft/monad-rpc/src/cli.rs` [#L63-L73](#)

The JSON-RPC server fully materializes and serializes responses before enforcing the response size cap, and it aggregates entire batches into memory before a final serialization pass. This creates a multiplicative memory footprint that an unauthenticated client can exploit to exhaust memory and crash the process under default settings.

- Batch aggregation: `rpc_handler()` runs each request concurrently via `futures::future::join_all(...)`, collects `(id, Result<Box<RawValue>, JsonRpcError>)` pairs, and converts them into a `Vec<Response>`. Each `Response` already contains a pre-serialized `Box<RawValue>` for the per-item result.
- Post-serialization size check: After the batch is fully constructed, the server calls `serde_json::value::to_raw_value(&response)` to serialize the entire `ResponseWrapper` into a second large `RawValue`, and only then checks its size: see `response_raw_value` and the subsequent length check in `rpc_handler()`. This duplicates memory usage at the worst possible time (the entire batch), so the size cap rejects only after the peak allocation.
- Amplification surface: `eth_getLogs` produces large outputs and lacks per-endpoint concurrency gating. The RPC wrapper simply fetches and returns `Vec<MonadLog>` in `monad_eth_getLogs()`. While `eth_call` / `eth_estimateGas` have a permit-based limiter, there is no equivalent guard for `eth_getLogs` in the `eth_getLogs` wrapper.
- Defaults exacerbate the issue: the CLI defaults permit very large batches and a relatively high response size cap (`batch_request_limit = 5000`, `max_response_size = 25_000_000`) in `cli`. With `N=5000` and per-item results of even ~200 KB, the batch holds ~1 GB of per-item `RawValue` buffers before creating the final batch `RawValue`. With ~1.5 MB per item (wide `eth_getLogs` filters), this easily reaches many GB, resulting in OOM or severe swapping/CPU thrash before the size cap is applied.

The same late size check also applies to `ResponseWrapper::Single`. Combined with costly response builders like `monad_eth_feeHistory()`, a single request can allocate very large structures (e.g., reward matrices sized by `block_count * len(reward_percentiles)`) and only then be rejected after serialization.

Worst-case outcome: A single crafted request or one batch forces the server to build large in-memory results and then serialize the entire wrapper. Memory balloons and the process is terminated by the OOM killer or becomes unresponsive, denying service to honest users. This is an implementation-specific weakness, not a generic network flood, and is exploitable on default config.

🔗 Recommended mitigation steps

Option A (preferred): stream responses with budgeting

- Replace final `serde_json::value::to_raw_value(&response)` with a streaming JSON encoder for `ResponseWrapper` that writes to the HTTP body while tracking a byte budget.
- Abort emission early with an error once the per-request or aggregate batch budget is exceeded.
- This avoids constructing a second massive `RawValue`.

Option B (quick hardening/defense in depth): cap batch and pre-reject oversized batches

[View detailed Proof of Concept](#)

🔗 [M-O4] Incorrect Write Position in Block Device Trim Operation

Submitted by [inh3l](#)

`monad/category/async/storage_pool.cpp` [#L258-L320](#)

The `chunk::try_trim_contents()` method for block devices contains a critical bug in handling partial page preservation during trim operations. When a trim boundary falls within a disk page (i.e., `remainder > 0`), the code:

1. Reads the partial page from the original offset (`range[0]`)
2. Advances `range[0]` by `DISK_PAGE_SIZE` to skip the partial page in the trim operation
3. But then incorrectly writes the modified buffer back to the advanced `range[0]` instead of the original offset

🔗 Current Buggy Implementation

```
if (remainder > 0) {
    range[0] += DISK_PAGE_SIZE; // Advance for trim
    range[1] -= DISK_PAGE_SIZE;
}
// ... trim operation happens ...
if (remainder > 0) {
    // @audit Writing to advanced range[0] instead of original position
    MONAD_ASSERT_PRINTF(
        -1 != ::pwrite(
            write_fd_,
            buffer,
            DISK_PAGE_SIZE,
            static_cast<off_t>(range[0])), // Wrong offset!
        "failed due to %s",
        strerror(errno));
}
```

This writes the preserved data fragment to the next disk page, corrupting whatever data was stored there.

🔗 Recommended mitigation steps

Store the original offset before modification and use it for the write operation.

🔗 [M-05] Typed receipt encoding does not conform to standard Ethereum RPC format

Submitted by [Evo](#)

`bft/monad-rpc/src/handlers/debug.rs` [#L126](#)

🔗 Root Cause

The `debug_getRawReceipts` RPC method in `bft/monad-rpc/src/handlers/debug.rs` violates the EIP-2718 specification by using incorrect encoding for typed transaction receipts. The method uses plain RLP encoding (`r.encode()`) instead of EIP-2718 encoding (`r.encode_2718()`), resulting in receipts that are missing their required transaction type prefix byte.

🔗 Technical Details

Current Implementation:

<https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-rpc/src/handlers/debug.rs#L126>

```
// In bft/monad-rpc/src/handlers/debug.rs, monad_debug_getRawReceipts function
let encode_receipts = |receipts: Vec<ReceiptEnvelope<alloy_primitives::Log>>| {
    let receipts = receipts
        .into_iter()
        .map(|r| {
            let mut res = Vec::new();
            r.encode(&mut res); // ← ISSUE: Using plain RLP encoding
            hex::encode(&res)
        })
        .collect();
    Ok(MonadDebugGetRawReceiptsResult { receipts })
}
```

```
};
```

Correct Implementation (as seen in `debug_getRawTransaction` in the same file L156):

<https://github.com/code-423n4/2025-09-monad/blob/main/bft/monad-rpc/src/handlers/debug.rs#L156>

```
let mut res = Vec::new();
let tx: TxEnvelope = tx.into();
tx.encode_2718(&mut res); // ← CORRECT: Using EIP-2718 encoding
```

The EIP-2718 Violation

According to the EIP-2718 specification:

“Receipt is either TransactionType || ReceiptPayload or LegacyReceipt”

And critically:

“The TransactionType of the receipt MUST match the TransactionType of the transaction with a matching Index.”

The specification defines that for typed transactions:

- `TransactionType` is a positive unsigned 8-bit number between 0 and 0x7f
- `ReceiptPayload` is an opaque byte array whose interpretation depends on the `TransactionType`
- The format MUST be: `TransactionType || ReceiptPayload` where `||` is the concatenation operator

Example of the violation for an EIP-1559 transaction:

Component	Expected (EIP-2718 compliant)	Actual (Current Implementation)
Transaction	<code>0x02 \\ \\ TransactionPayload</code>	<code>0x02 \\ \\ TransactionPayload</code> ✓
Receipt	<code>0x02 \\ \\ ReceiptPayload</code>	<code>RLP([status, gas, bloom, logs])</code> X

The receipt is missing the critical `0x02` `TransactionType` byte prefix, violating the MUST requirement that “The `TransactionType` of the receipt MUST match the `TransactionType` of the transaction.”

Dependency Context

The project uses the `alloy-consensus` crate (v0.8.3) which provides the `ReceiptEnvelope` type. This type correctly implements the `Encodable2718` trait in `alloy-consensus-0.8.3/src/receipt/envelope.rs`:

```
// From alloy-consensus-0.8.3/src/receipt/envelope.rs
impl Encodable2718 for ReceiptEnvelope {
    fn encode_2718(&self, out: &mut dyn BufMut) {
        match self.type_flag() {
            None => {} // Legacy receipts - no prefix
            Some(ty) => out.put_u8(ty), // Typed receipts - adds type byte
        }
        self.as_receipt_with_bloom().unwrap().encode(out);
    }
}
```

The `encode_2718()` method automatically handles all receipt types:

- Legacy (no prefix, remains as `rlp([status, cumulativeGasUsed, logsBloom, logs])`)
- EIP-2930 (0x01 prefix)

- EIP-1559 (0x02 prefix)
- EIP-4844 (0x03 prefix)
- EIP-7702 (0x04 prefix)

🔗 Impact

- Malformed receipts could cause incorrect state verification or proof generation in systems relying on proper receipt encoding, and the tools and clients expecting EIP-2718 compliant receipts will fail to parse them correctly.
- Violates EIP-2718's MUST requirements - the spec requires matching TransactionType between transactions and receipts, and the method documentation falsely claims to return "EIP-2718 binary-encoded receipts", as external clients cannot differentiate receipt types as the first byte should be in range [0, 0xf] for typed transactions but instead contains RLP data.

🔗 Recommended mitigation steps

Replace the plain RLP encoding with EIP-2718 encoding in the `debug_getRawReceipts` function:

```
// In bft/monad-rpc/src/handlers/debug.rs, line ~49
let encode_receipts = |receipts: Vec<ReceiptEnvelope<alloy_primitives::Log>>| {
    let receipts = receipts
        .into_iter()
        .map(|r| {
            let mut res = Vec::new();
            r.encode_2718(&mut res); // ← FIX: Use encode_2718 instead of encode
            hex::encode(&res)
        })
        .collect();
    Ok(MonadDebugGetRawReceiptsResult { receipts })
};
```

🔗 [M-06] Vote timer callback uses potentially stale timer round instead of actual vote round, causing vote misrouting

Submitted by [ZeroEx](#), also found by [JuggerNaut63](#) and [minato7namikazi](#)

`bft/monad-consensus-state/src/lib.rs` [#L1061](#)

In `handle_vote_timer()`, when a vote timer fires, the callback extracts the stored vote but uses the timer's round parameter instead of the vote's actual round when calling `send_vote_and_reset_timer()`:

```
pub fn handle_vote_timer(
    &mut self,
    round: Round, // ← Timer's round
) -> Vec<ConsensusCommand<...>> {
    let Some(OutgoingVoteStatus::VoteReady(v)) = self.consensus.scheduled_vote else {
        self.consensus.scheduled_vote = Some(OutgoingVoteStatus::TimerFired);
        return vec![];
    };

    self.send_vote_and_reset_timer(round, v) // ← Uses timer's round, not v.round
}
```

In `send_vote_and_reset_timer()`, vote recipients are computed from the passed round parameter:

```
let next_leader = get_leader(round + Round(1)); // ← Derives from parameter
let current_leader = get_leader(round); // ← Derives from parameter
```

Vote timers are scheduled one round ahead when sending votes:

```
cmds.push(ConsensusCommand::ScheduleVote {
    round: round + Round(1), // ← Schedules for next round
    duration: vote_pace,
```

```
});
```

Trigger Scenario

1. Node votes on round R, schedules timer for round R+1
2. Before timer fires, node rapidly advances through R+1 → R+2 (via QC messages and/or consecutive proposals)
3. Node processes proposal for R+2, updates `scheduled_vote = VoteReady(v_R+2)`
4. Stale timer for R+1 fires
5. `handle_vote_timer(R+1)` extracts vote `v_R+2` but passes `round = R+1` to `send_vote_and_reset_timer()`

Impact

Vote Misrouting:

- Advance by 2 rounds: Vote sent to current leader but not next leader
 - Next-round leader (R+3) never receives vote for R+2
 - Slows QC(R+2) formation → increases timeout risk
- Advance by 3+ rounds (*more severe*): Vote sent to neither current nor next leader
 - Current leader (R+N) doesn't receive vote for R+N
 - Completely blocks QC formation for round R+N
 - Forces timeout and TC generation
 - Severely degrades liveness

Cascading Timer Drift:

- Next timer scheduled for round R+2 (should be R+3)
- Subsequent stale timers compound the misrouting

Can occur during fast network conditions, burst QC message processing, or catch-up after brief partitions, where rapid round advancement causes stale vote timers to fire after consensus has progressed multiple rounds ahead.

[View detailed Proof of Concept](#)

[M-07] Consensus validator accepts blocks with mismatched basefee and execution `basefeepergas`

Submitted by [lian886](#), also found by [OxAsen](#), [fromeo_016](#), [Jorgect](#), and [QuestceQuecest](#)

- `bft/monad-eth-block-validator/src/lib.rs` [#L182](#)
- `bft/monad-eth-block-validator/src/lib.rs` [#L405](#)
- `bft/monad-eth-tpool/src/pool/mod.rs` [#L357](#)
- `bft/monad-updaters/src/txpool.rs` [#L320](#)
- `bft/monad-state/src/consensus.rs` [#L360](#)
- `monad/category/execution/monad/validate_monad_block.cpp` [#L40](#)

`validate_block_header` only re-validates ancillary `ProposedEthHeader` fields (transaction tree, mix hash,

timestamp, etc.) and never ensures that the execution header's `base_fee_per_gas` matches the consensus header's `base_fee`. During block validation the subsequent fee checks inside `validate_block_body` (`bft/monad-eth-block-validator/src/lib.rs:405`) therefore rely solely on `header.base_fee.unwrap_or(monad_tfm::base_fee::PRE_TFM_BASE_FEE)`, ignoring the actual execution base fee.

Because the txpool builds the execution header and consensus header via independent data paths—`EthTxPool::create_proposal` fixes `base_fee_per_gas` when constructing the `ProposedEthHeader` (`bft/monad-eth-txpool/src/pool/mod.rs:357`), while the consensus side writes independently supplied `base_fee` /trend/moment values into the `ConsensusBlockHeader` (`bft/monad-updaters/src/txpool.rs:320` → `bft/monad-state/src/consensus.rs:360`)—a malicious proposer can deliberately diverge these numbers. The validator will still accept the block so long as `header.base_fee` is low enough for the static per-transaction fee checks.

Once the block reaches execution, the engine relies on `execution_inputs.base_fee_per_gas` and will detect the inconsistency only after consensus acceptance: `static_validate_consensus_header` for `MonadConsensusBlockHeaderV2` raises `BaseFeeMismatch` when the consensus `base_fee` differs from the execution header's `base_fee_per_gas` (`monad/category/execution/monad/validate_monadblock.cpp:40`). This late rejection causes a consensus/execution split: consensus finalizes a block that execution cannot run, and all max-fee transactions chosen between the forged `base_fee` and the true `base_fee_per_gas` will fail to cover gas, guaranteeing execution failure and divergence. For pre-TFM rounds, the validator even permits `base_fee == None`, silently substituting the `PRE_TFM_BASE_FEE` constant while execution still charges the high `base_fee_per_gas`, making the exploit possible regardless of TFM activation.

On a live network every replica would receive the same finalized header and hit the identical `BaseFeeMismatch`, so executors everywhere halt while consensus has already voted the block in. That leaves the ledger stuck (or nodes cycling if restarted), producing a persistent consensus/exec split until operators intervene—explaining why enforcing the match during Rust-side validation is critical.

🔗 Recommended mitigation steps

Add an explicit consistency check inside `validate_block_header`: when `header.base_fee` is `Some`, require equality with `execution_inputs.base_fee_per_gas`. When it is `None`, enforce that `execution_inputs.base_fee_per_gas` equals `monad_tfm::base_fee::PRE_TFM_BASE_FEE`. Reject any block where these fields diverge so the validator shares execution's invariant before the block is accepted.

[View detailed Proof of Concept](#)

🔗 Low Risk and Informational Issues

For this audit, 21 QA reports were submitted by wardens compiling low risk and informational issues. The [QA report highlighted below](#) by Almanax received the top score from the judge. 18 Low-severity findings were also submitted individually, and can be viewed [here](#).

The following wardens also submitted QA reports: [Oxbrett8571](#), [Oxki](#), [OxPhantom](#), [Auditor Nate](#), [blaze18](#), [codegpt](#), [emerald7017](#), [foxb868](#), [K42](#), [KeccakCrew](#), [mbuba666](#), [nOm4d1c](#), [b34r](#), [RadiantLabs](#), [rics](#), [SAQ](#), [Sathish9098](#), [Satyam Sharma](#), [TheCarrot](#), [vangrim](#), and [yongskiws](#).

🔗 [L-01] HTTP Host label unsanitized → unbounded metrics cardinality (Invariant: OpSec)

Description: The Prometheus/OpenTelemetry attributes take `server.address` directly from the Host header.

Impact: Info/ops (cardinality explosion in metrics backend).

Evidence: `bft/monad-rpc/src/metrics.rs:95-106` (`conn_info.host()` split to `server.address` / `server.port`).

Instances: n/a

Guards/Assumptions: None.

Recommendation: Normalize/sanitize Host to a canonical set, or record connection local address only.

Function: `bft/monad-rpc/src/metrics.rs::attributes_from_request()`. Replace

`server.address` / `server.port` extraction from `conn_info.host()` with a canonicalized value (e.g., `server.local_address` from the bound socket) and a low-cardinality `server.port`.

Minimal diff (replace host-derived labels with local bind address/port):

```
diff --git a/bft/monad-rpc/src/metrics.rs b/bft/monad-rpc/src/metrics.rs
@@ fn attributes_from_request(req: &Request, conn_info: &ConnInfo) -> Attributes {
-   let (host, port) = parse_host(conn_info.host());
-   attrs.insert("server.address", host);
-   attrs.insert("server.port", port);
+   let local = conn_info.local_addr();
+   attrs.insert("server.local_address", local.ip().to_string());
+   attrs.insert("server.port", local.port().to_string());
}
```

Why Low (not Info): High-cardinality metrics can severely degrade observability backends and node performance under load.

Regression notes: Cardinality will drop; retain observability by recording `server.local_address` and method/path tags.

Default-node path: any external client can vary the `Host` header against the default RPC listener; current labeling records `server.address` unnormalized, inflating series cardinality without additional flags.

🔗 [L-02] JSON-RPC returns internal error details → information disclosure (Invariant: OpSec)

Description: Internal `ChainStateError::{Archive,Triedb}` messages are included in error text.

Impact: Info (implementation leakage).

Evidence: `bft/monad-rpc/src/jsonrpc.rs:271-286, 411-444` (formats "Archive error: {e}").

Instances: n/a

Guards/Assumptions: None; logs also record details.

Recommendation: Return generic messages; keep details in server logs only. Minimal rewrite: in `bft/monad-rpc/src/jsonrpc.rs::ChainStateResultExt::to_jsonrpc_result`, map `Archive(_)` and `Triedb(_)` to `JsonRpcError::internal_error("Archive error")` / `JsonRpcError::internal_error("Triedb error")` while logging details with `tracing::error!`. Also update `archive_to_jsonrpc_error` and `From<monad_archive::prelude::Report>` to log `error!(...)` and return generic `JsonRpcError::internal_error("Archive error: {message}")`.

Minimal mapping and logging snippet (redacted client errors, detailed logs retained):

```
match err {
  ChainStateError::Archive(e) => {
    tracing::error!(error = ?e, "archive error");
    return Err(JsonRpcError::internal_error("Archive error"));
  }
  ChainStateError::Triedb(e) => {
    tracing::error!(error = ?e, "triedb error");
    return Err(JsonRpcError::internal_error("Triedb error"));
  }
  // ... other variants
}
```

Why Low (not Info): Increases attack surface by leaking environment/state; also confuses operators and clients.

Regression notes: Client messages remain spec-compatible; internal details preserved in logs for operators.

Default-node path: JSON-RPC error responses are reachable on a default node without special flags (e.g., archive disabled); responses embed internal error text that leaks environment/storage details.

🔗 Disclosures

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by

a knowledgeable security researcher and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.

[Top](#)

[Twitter](#) | [Discord](#) | [GitHub](#) | [Media kit](#) | [Terms](#) | [Privacy](#)